Redes de Datos

Tema 6: Protocolos de Transporte

Que vamos a ver…

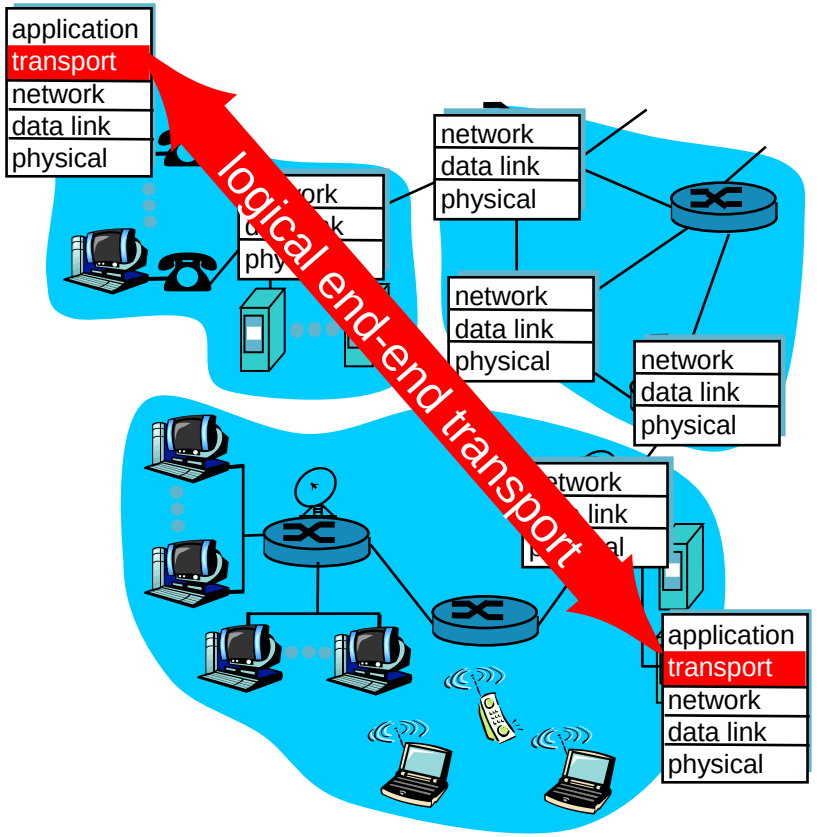
En la clase de hoy:

- Protocolos y servicios de transporte.
- Demultiplexación.
- UDP y TCP.
- Control de flujo y congestión.
- Técnica de “Ventana Deslizante”.
- Administración de conexión en TCP.
- Control de congestión en TCP.

Introducción

Protocolos y Servicios de Transporte

- Proveen ***comunicación lógica*** entre procesos aplicación corriendo en diferentes hosts.
- Los protocolos de transporte ***corren en sistemas terminales*** (computadoras, no equipos como routers o switches).
  - *Lado Transmisor:* ***divide el mensaje de la aplicación en segmentos***, y los pasa a la capa de red.
  - *Lado Receptor:* ***re-ensambla los segmentos del*** mensaje , y lo pasa a la capa aplicación.
- Hay más de un protocolo de transporte disponible para las aplicaciones.
  - Internet: TCP y UDP.



Protocolos de Capa Transporte en Internet

TCP (Transfer Control Protocol)	UDP (User Datagram Protocol)
Control de Congestión	Entrega No Confiable
Control de Flujo	Entrega (tal vez) desordenada
Establecimiento de Conexión	Técnica “Mejor Esfuerzo” (Best-Effort)

NO OFRECEN:

- Garantías de retardo.
- Garantías de ancho de banda.

Básicamente porque no es posible ofrecerlo basándose en los servicios de IP.

Capa de Red vs Capa de Transporte

Capa de Red	*Capa Transporte*
• Es la encargada de la comunicación lógica **entre hosts**.	• Es la encargada de la comunicación lógica **entre procesos**. • Mejora los servicios de la capa de red.

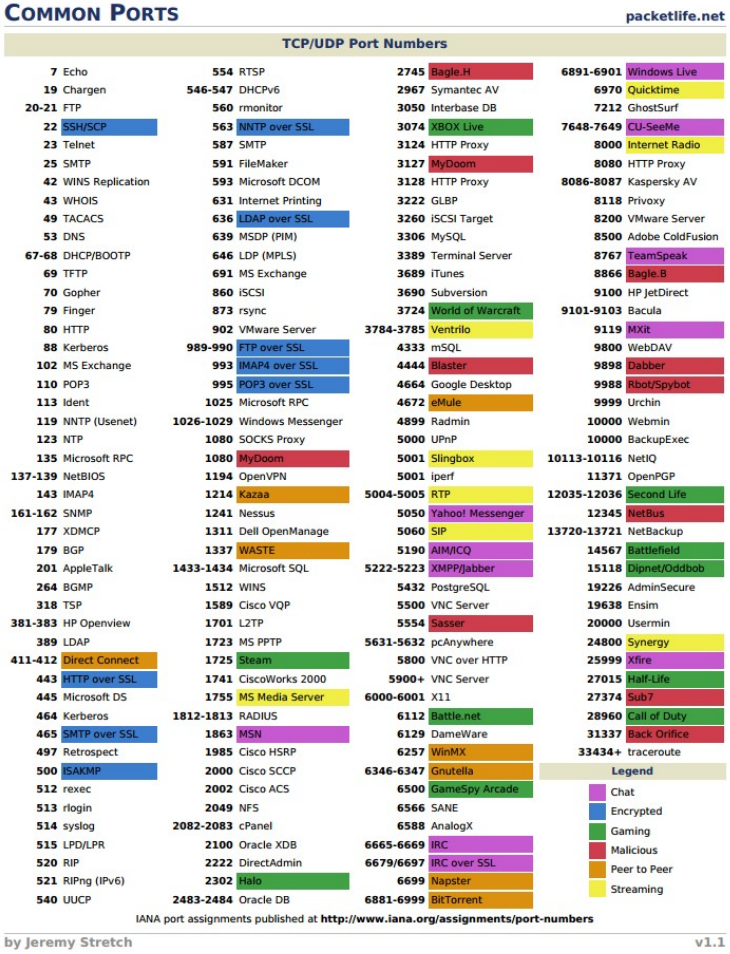
Encapsulado

¿Cómo trabaja el encapsulado en esta capa?

- El host recibe datagramas IP.
  - Cada datagrama tiene dirección IP fuente y dirección IP destino.
  - Cada datagrama lleva 1 segmento de la capa transporte.
  - Cada segmento tiene números de puerto fuente y destino (recordar que hay números de puerto conocidos para aplicaciones específicas).
- El host usa direcciones IP y números de puertos para conducir un segmento al socket apropiado.

Formato de segmento TCP/UDP

Listado de Puertos Comunes



COMMON PORTS packetlife.net

TCP/UDP Port Numbers

7	Echo	554	RTSP	2745	Bagle.H	6891-6901	Windows Live
19	Chargen	546-547	DHCPv6	2967	Symantec AV	6970	Quicktime
20-21	FTP	560	rmonitor	3050	Interbase DB	7212	GhostSurf
22	SSH/SCP	563	NNTP over SSL	3074	XBOX Live	7648-7649	CU-SeeMe
23	Telnet	587	SMTP	3124	HTTP Proxy	8000	Internet Radio
25	SMTP	591	FileMaker	3127	MyDoom	8080	HTTP Proxy
42	WINS Replication	593	Microsoft DCOM	3128	HTTP Proxy	8086-8087	Kaspersky AV
43	WHOIS	631	Internet Printing	3222	GLBP	8118	Privoxy
49	TACACS	636	LDAP over SSL	3260	iSCSI Target	8200	VMware Server
53	DNS	639	MSDP (PIM)	3306	MySQL	8500	Adobe ColdFusion
67-68	DHCP/BOOTP	646	LDP (MPLS)	3389	Terminal Server	8767	TeamSpeak
69	TFTP	691	MS Exchange	3689	iTunes	8866	Bagle.B
70	Gopher	860	iSCSI	3690	Subversion	9100	HP JetDirect
79	Finger	873	rsync	3724	World of Warcraft	9101-9103	Bacula
80	HTTP	902	VMware Server	3784-3785	Ventrilo	9119	MXit
88	Kerberos	989-990	FTP over SSL	4333	mSQL	9800	WebDAV
102	MS Exchange	993	IMAP4 over SSL	4444	Blaster	9898	Dabber
110	POP3	995	POP3 over SSL	4664	Google Desktop	9988	Rbot/Spybot
113	Ident	1025	Microsoft RPC	4672	eMule	9999	Urchin
119	NNTP (Usenet)	1026-1029	Windows Messenger	4899	Radmin	10000	Webmin
123	NTP	1080	SOCKS Proxy	5000	UPnP	10000	BackupExec
135	Microsoft RPC	1080	MyDoom	5001	Slingbox	10113-10116	NetIQ
137-139	NetBIOS	1194	OpenVPN	5001	iperf	11371	OpenPGP
143	IMAP4	1214	Kazaa	5004-5005	RTP	12035-12036	Second Life
161-162	SNMP	1241	Nessus	5050	Yahoo! Messenger	12345	NetBus
177	XDMCP	1311	Dell OpenManage	5060	SIP	13720-13721	NetBackup
179	BGP	1337	WASTE	5190	AIM/ICQ	14567	Battlefield
201	AppleTalk	1433-1434	Microsoft SQL	5222-5223	XMPP/Jabber	15118	Dipnet/Oddbob
264	BGMP	1512	WINS	5432	PostgreSQL	19226	AdminSecure
318	TSP	1589	Cisco VQP	5500	VNC Server	19638	Ensim
381-383	HP Openview	1701	L2TP	5554	Sasser	20000	Usermin
389	LDAP	1723	MS PPTP	5631-5632	pcAnywhere	24800	Synergy
411-412	Direct Connect	1725	Steam	5800	VNC over HTTP	25999	Xfire
443	HTTP over SSL	1741	CiscoWorks 2000	5900+	VNC Server	27015	Half-Life
445	Microsoft DS	1755	MS Media Server	6000-6001	X11	27374	Sub7
464	Kerberos	1812-1813	RADIUS	6112	Battle.net	28960	Call of Duty
465	SMTP over SSL	1863	MSN	6129	DameWare	31337	Back Orifice
497	Retrospect	1985	Cisco HSRP	6257	WinMX	33434+	traceroute
500	ISAKMP	2000	Cisco SCCP	6346-6347	Gnutella		
512	rexec	2002	Cisco ACS	6500	GameSpy Arcade		
513	rlogin	2049	NFS	6566	SANE		
514	syslog	2082-2083	cPanel	6588	AnalogX		
515	LPD/LPR	2100	Oracle XDB	6665-6669	IRC		
520	RIP	2222	DirectAdmin	6679/6697	IRC over SSL		
521	RIPng (IPv6)	2302	Halo	6699	Napster		
540	UUCP	2483-2484	Oracle DB	6881-6999	BitTorrent		

Legend

- Chat
- Encrypted
- Gaming
- Malicious
- Peer to Peer
- Streaming

IANA port assignments published at **http://www.iana.org/assignments/port-numbers**

by Jeremy Stretch v1.1

- 20-21: FTP
- 22: SSH
- 23: Telnet
- 25: SMTP
- 110: POP3
- 143: IMAP4
- 1812: RADIUS
- 3128/8080: proxy HTTP

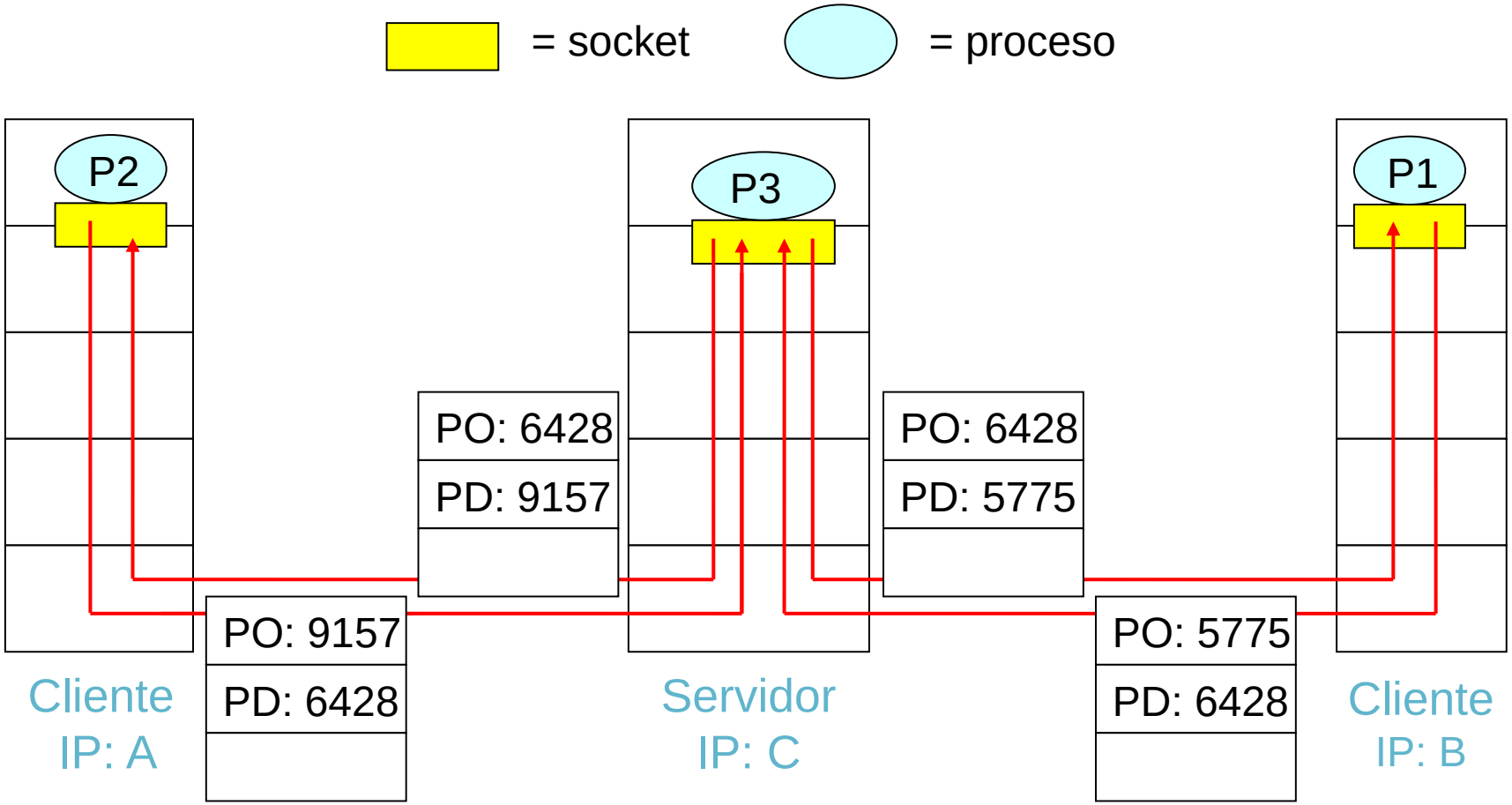
…

http://en.wikipedia.org/wiki/List_of_TCP_and_UDP_port_numbers

Demultiplexación Sin Conexión (UDP)

- Creamos sockets con números de puerto.
- El Socket UDP queda identificado por dupla:
  - Dirección IP de destino.
  - Número de puerto de destino.
- Cuando un host recibe un segmento UDP:
  - Chequea número de puerto destino en segmento.
  - Dirige el segmento UDP al socket con ese número de puerto.
- **Datagramas IP con direcciones IP y/o números de puerto origen diferentes pueden ser dirigidos al mismo socket.**

Demultiplexación Sin Conexión (UDP)

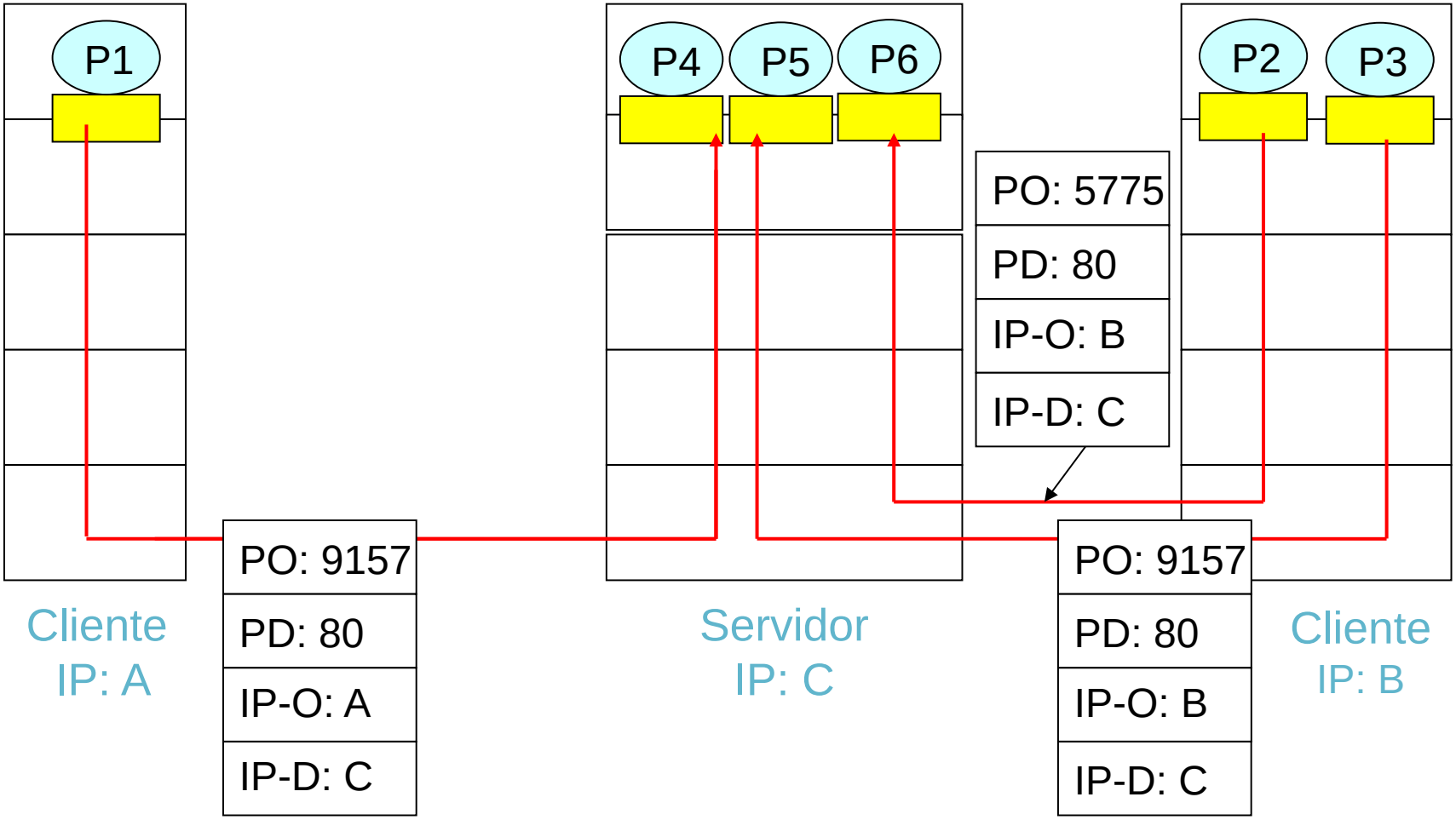


PO: el Puerto de Origen, provee la “dirección de retorno”.

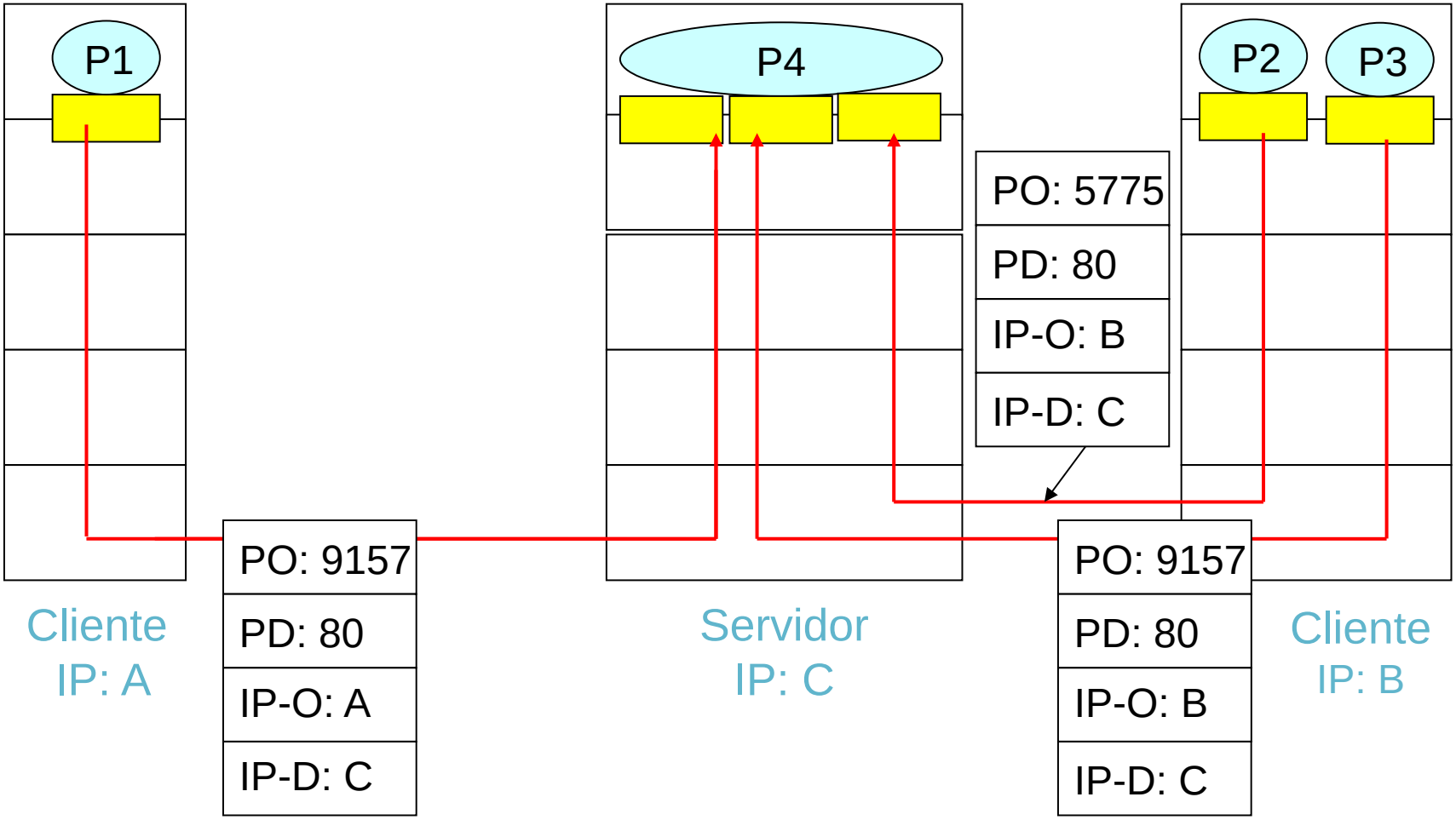
Demultiplexión Orientada a la Conexión (TCP)

- Sockets TCP queda definido por 4-tupla:
  - Dirección IP Origen.
  - Número de puerto Origen.
  - Dirección IP Destino.
  - Número de puerto Destino.
- El Host Receptor usa los cuatro valores para dirigir los segmentos al socket apropiado.
- Un Host Servidor puede soportar muchos sockets TCP simultáneos. Cada socket es identificado por su 4-tupla propia.
- Servidores web tiene sockets diferentes por cada cliente conectado. HTTP no-persistente tendrá diferentes sockets por cada pedido.

Demultiplexión Orientada a la Conexión (TCP)



Demultiplexión Orientada a la Conexión (TCP)



UDP: User Datagram Protocol

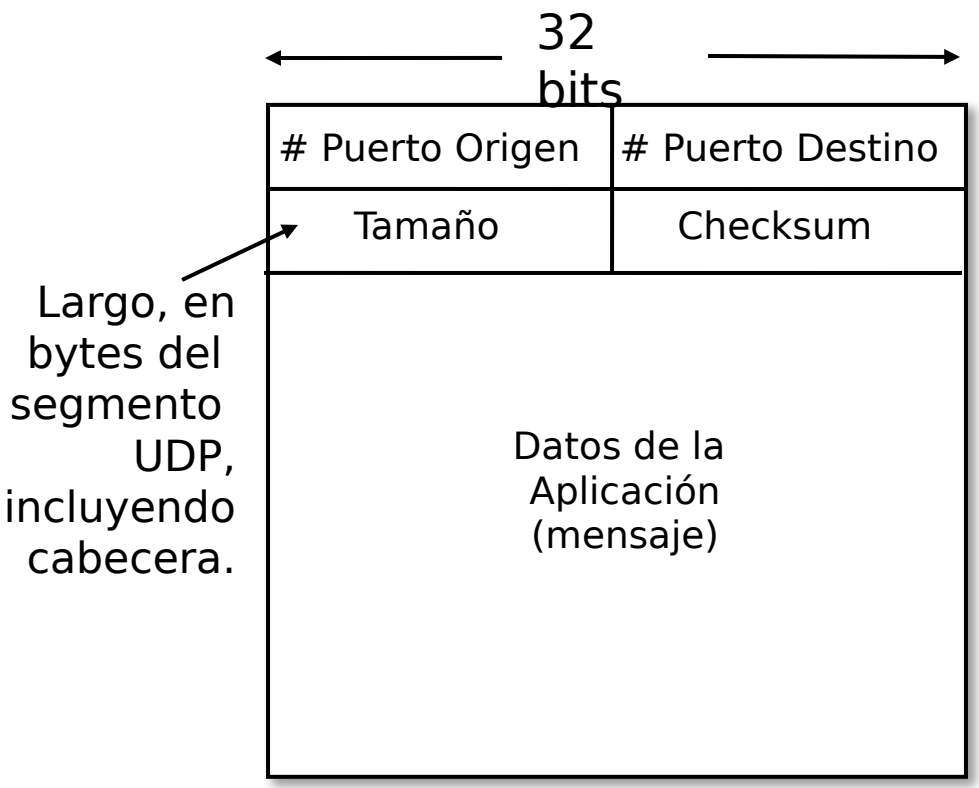
- Protocolo Internet de transporte “sin adicionales”.
- Servicio de “mejor esfuerzo”, un segmento UDP puede ser:
  - Perdido.
  - Entregado a la aplicación fuera de orden.
- ***Sin conexión:***
  - **No hay handshaking** (apretón de manos) entre servidor y receptor.
  - Cada segmento es manejado en forma independiente de los otros.

Por qué existe UDP?

- No requiere establecimiento de conexión, lo cual agrega retardo.
- Es simple, no se requiere mantener estado en el transmisor y el receptor.
- Pequeño segmento de encabezado, por lo cual tiene menor overhead.
- No hay control de congestión, UDP puede transmitirse tan rápido como se desee.

UDP: User Datagram Protocol

- A menudo es usado por flujos (streaming) multimedia en aplicaciones debido a:
  - Tolerancia a pérdida.
  - Sensibilidad a la tasa.
- Otros usos de UDP:
  - DNS (Domain Name System).
  - SNMP (Simple Network Management Protocol).
- Transferencia confiable sobre UDP:
  - Agrega confiabilidad en la capa aplicación.
  - Recuperación de errores específicos según la aplicación.



Formato segmento UDP

Checksum (Suma de Verificación)

<u>Objetivo</u>: Detección de errores
(por ejemplo, bits cambiados en segmentos transmitidos).

Transmisor

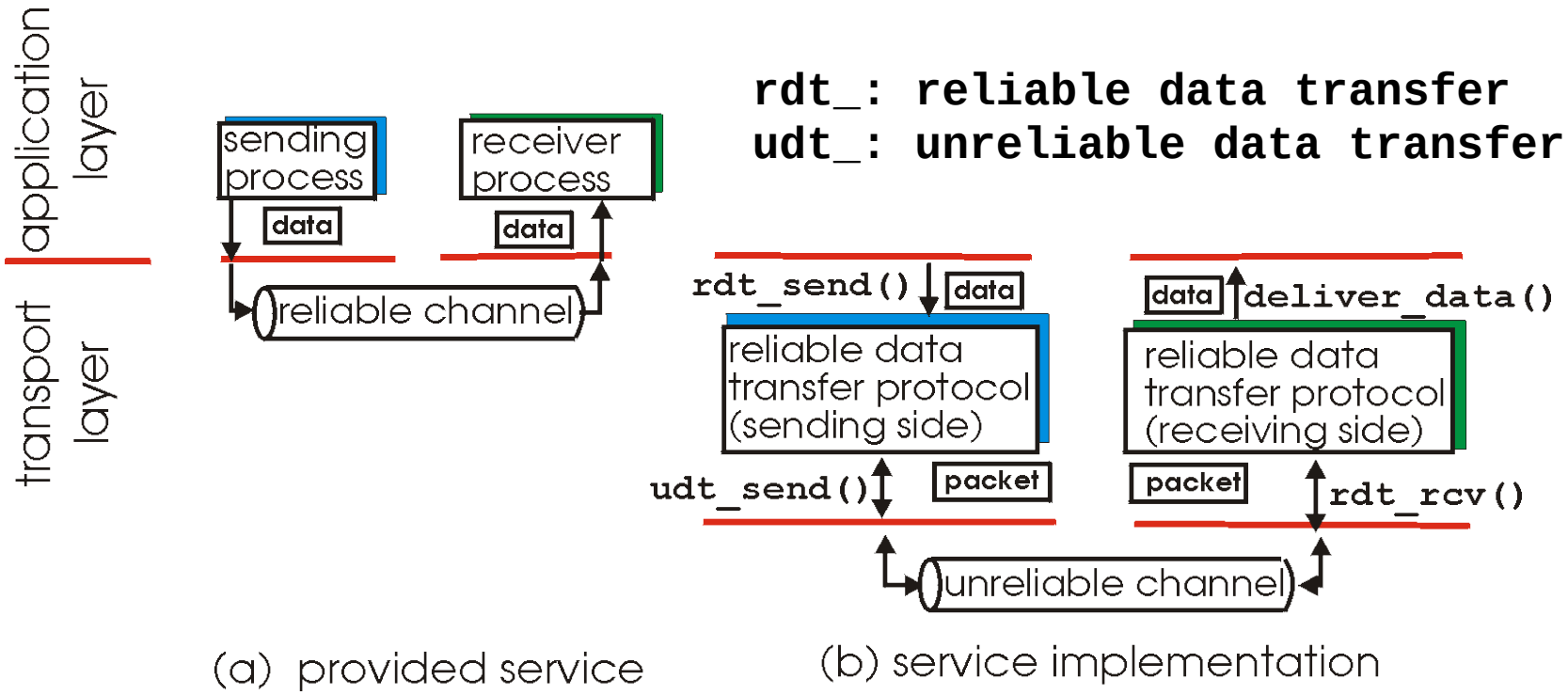
- Trata el contenido de cada segmento como una secuencia de enteros de 16 bits.
- Suma del contenido del segmento y luego toma el complemento a 1.
- El transmisor pone el valor calculado en el campo checksum del datagrama UDP.

Receptor

- Calcula el checksum del segmento recibido.
- Verifica si el valor calculado corresponde al valor de checksum recibido en el campo:
  - **Si Coincide**: No hay error.
  - **No Coincide**: Error detectado!!!

Principios de Transferencia Confiable de Datos

- Es importante en capas de aplicación, transporte y enlace de datos.
- Está en la lista de los 10 tópicos más importantes sobre redes.



- Las características del canal no-confiable determinarán la complejidad del protocolo de datos confiable (reliable data transfer - rdt).

Principios de Transferencia Confiable de Datos

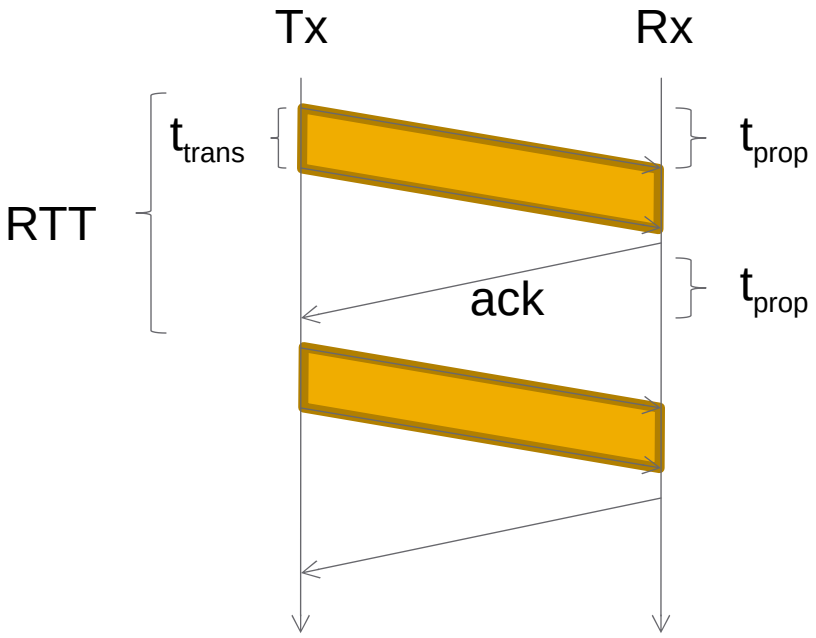
- Sobre canal confiable, no hace falta nada mas que transmitir y recibir.
- Sobre canal no confiable, hace falta verificar los datos y avisar si llegaron bien o mal.
  - Hay que coordinar el estado de ambos extremos, transmisor y receptor, mediante la realimentación (Acuse de Recibo).
    - Si llega bien, ACK (“Acknowldegment” o “Reconocimiento”) y se continua.
    - Si llega mal, NAK (“Negative Acknowldegment” o “Reconocimiento Negativo”) y se retransmite.

Principios de Transferencia Confiable de Datos

- El protocolo de transferencia confiable debe tener en cuenta otros escenarios posibles:
  - ¿Que pasa si el ACK o el NAK tiene errores?
    - El reenvío del último paquete puede introducir paquetes duplicados, que el receptor no puede diferenciar de nuevos datos; entonces, **se introducen los números de secuencia.**
  - ¿Qué pasa si no llega ni un ACK ni un NAK?
    - **Se implementan timers**, que esperen un RTT (“Round Trip Time” – “Tiempo de Viaje Redondo o de Ida y Vuelta”) mas un retardo de proceso en el receptor.

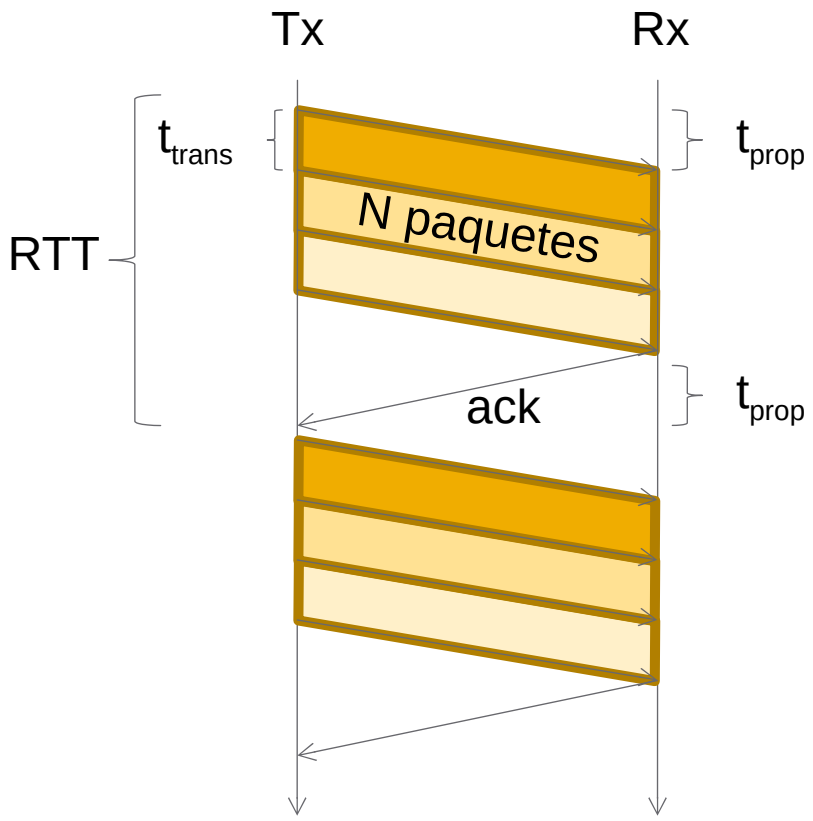
Protocolo de Parada y Espera

- Se envía un paquete, se espera una respuesta:
  - Si es **afirmativa**, se pasa al próximo paquete (**1 RTT**).
  - Si es **negativa** se retransmite el paquete original (**2 RTT**).
- Si calculamos la utilización del canal (tiempo en uso/tiempo total) para 100 Mbps y1Gbps, con un paquete de 1kByte y una distancia de de 8000 km:
  - 0,15% para 100 Mbps.
  - 0,015% para 1 Gbps.



$$RTT = t_{trans} + 2 \times t_{prop}$$

Protocolo de *"Pipeline"*



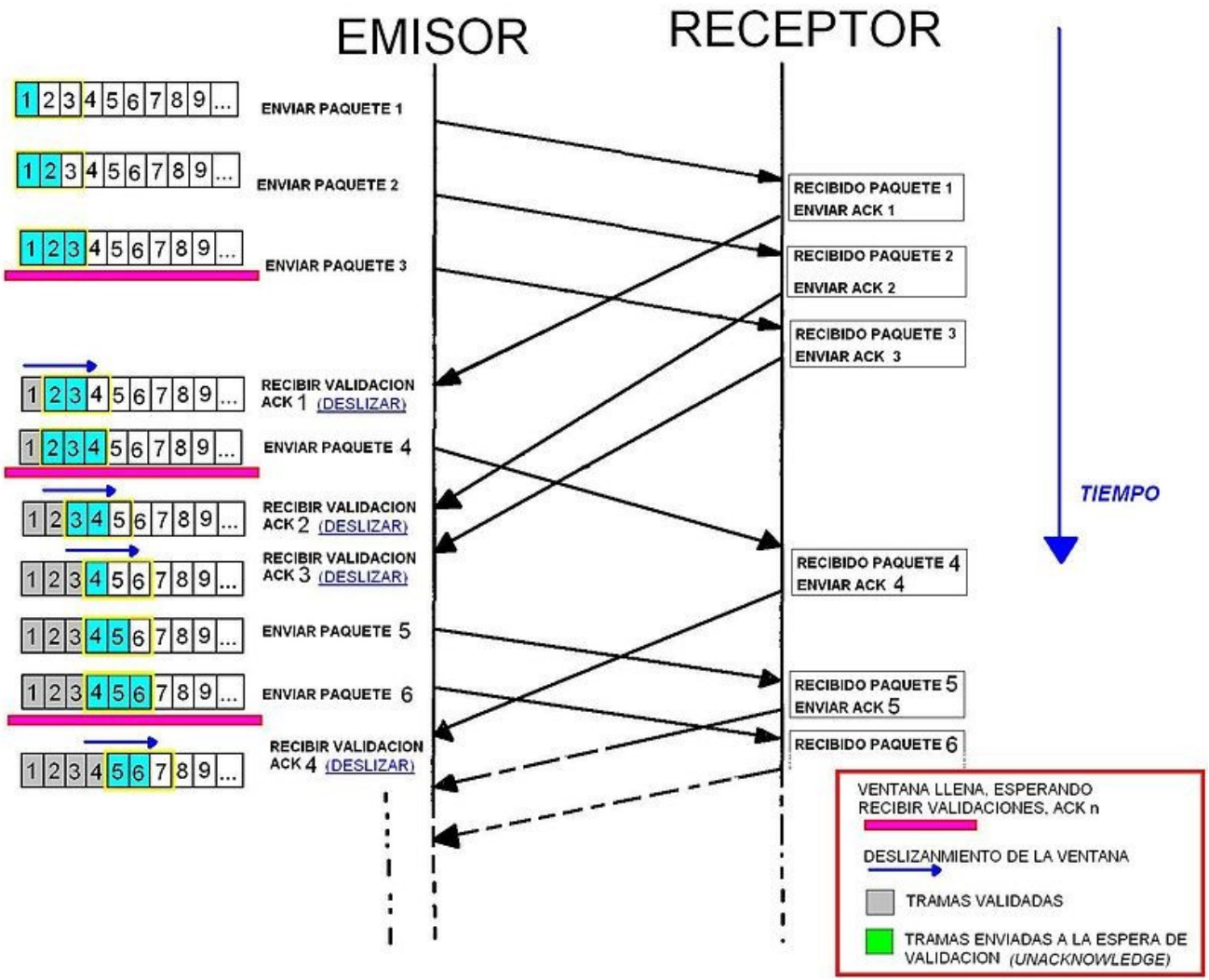
$$RTT = N \times t_{trans} + 2 \times t_{prop}$$

- Se envían varios paquetes sin esperar reconocimientos individuales. El RTT se compone de
  - $N \times t_{trans}$
  - $2 \times t_{prop}$
- Pero:
  - El emisor deberá tener una memoria temporal para los paquetes enviados y aún no reconocidos.
  - El receptor deberá tener una memoria temporal para la recepción de varios paquetes.
  - Se deberá llevar la cuenta de los paquetes mediante números de secuencia.

Técnica de “Ventana Deslizante”

- **Permite al transmisor transmitir múltiples segmentos** de información antes de comenzar la espera para que el receptor le confirme la recepción de los segmentos.
- **El transmisor deberá guardar en un buffer todas aquellas tramas enviadas y no validadas, por si necesitase retransmitirlas**.
- **El receptor posee una ventana de recepción**, **donde almacena** las tramas **temporalmente** en un buffer, hasta el momento que posea todas las esperadas de la secuencia completa, y así ordenarlas.
- **El receptor debe disponer de un buffer de igual tamaño que su ventana de recepción** para almacenar temporalmente las tramas hasta ordenarlas.
- **Existen 2 modos de trabajo** en función del tamaño de su ventana de recepción:
  - **Tamaño ventana recepción = 1:** Con lo cual la ventana de recepción dispone de **un** buffer. Sólo puede almacenar la trama que le llega en cada instante, es decir, debe recibir las tramas en la secuencia correcta, ya que no dispone de recursos para ordenarlas después. Impone al emisor la condición de transmitir siempre las tramas en secuencia.
  - **Tamaño ventana recepción > 1:** La ventana de recepción dispone de N buffers (N=tamaño ventana de recepción) que le permiten recibir hasta N tramas desordenadas, almacenarlas y proceder a su ordenamiento posterior. Le permite al emisor transmitir tramas desordenadas, tantas como quepan en los buffers del receptor.

Técnica de “Ventana Deslizante”



Recuperación de Errores

- Hay dos tipos, dependiendo como actúan cuando aparecen “errores”:
  - **Retroceder N:** Los paquetes enviados y no reconocidos luego de un tiempo, se reenvían en grupo. Los reconocimientos son acumulativos (un ACK para el paquete n-ésimo implica que los anteriores están bien).
  - **Reenvío Selectivo:** Se usa un timer por paquete y los paquetes son reconocidos individualmente.

Control de Congestión

- Informalmente, **congestión**, es cuando hay “demasiadas fuentes que envían **demasiados datos demasiado rápido** como para que la red lo maneje”.
- **Es distinto a Control de Flujo**.
- Manifestaciones:
  - Pérdidas de paquetes (buffer overflow en routers).
  - Grandes retardos (colas en los router).
- Uno de los problemas más críticos (“Top 10”).

¿Cómo podemos lidiar con este problema?

Estrategias para Control de Congestión

Los podemos clasificar en dos grupos amplios:

Extremo a Extremo

- No hay realimentación explícita de la red.
- La congestión es inferida desde las pérdidas y retardos observados por sistemas extremos.
- Es la estrategia usada por TCP.

Asistido por la Red

- Los routers proveen realimentación a los sistemas extremos (hosts).
- Bit único indicando congestión (SNA, DECbit, TCP/IP ECN, ATM).
- Explícitamente informa al transmisor la tasa el router puede soportar.

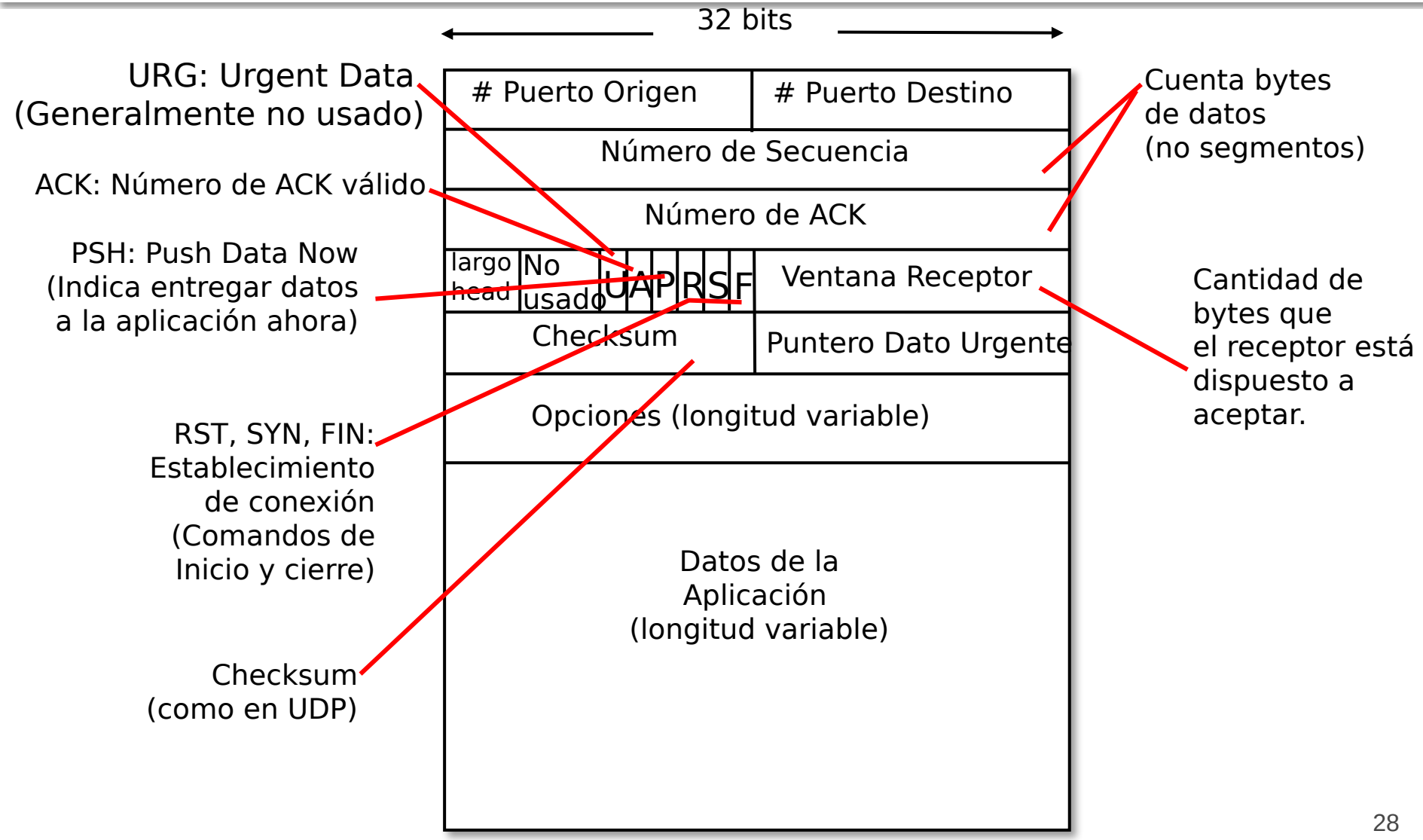
Generalidades TCP

- Transferencia **full duplex**:
  - Flujo de datos bi-direccionales en la misma conexión.
  - **Hay un tamaño máximo** de segmento (MSS: maximum segment size).
- Es **orientado a la conexión**:
  - **Usa handshaking** (intercambio de mensajes de control). Inicializa transmisor y receptor antes del intercambio de datos.
- Usa **control de flujo**:
  - El transmisor no sobrecargará al receptor con datos.
- **Punto a punto**: Un transmisor y un receptor.
- Es confiable, en orden del *flujo de bytes:*
  - No hay “límites de inicio/término de mensaje”.
- **Usa pipeline** (entubado):
  - El **tamaño de la ventana** TCP es **definido por el control de congestión y control de flujo.**
- **Usa buffers en el transmisor y en el receptor.**

Transferencia confiable de datos en TCP

- **TCP crea un servicio de transferencia confiable sobre el servicio no confiable de IP.**
- Usa segmentos en pipeline (entubado).
- ACKs acumulativos.
- TCP usa un timer de retransmisión único.
- Retransmisiones son activadas por:
  - Eventos timeout.
  - ACKs duplicados.
- Inicialmente consideremos un transmisor TCP simplificado:
  - Ignora ACKs duplicados.
  - Ignora control de flujo y control de congestión.

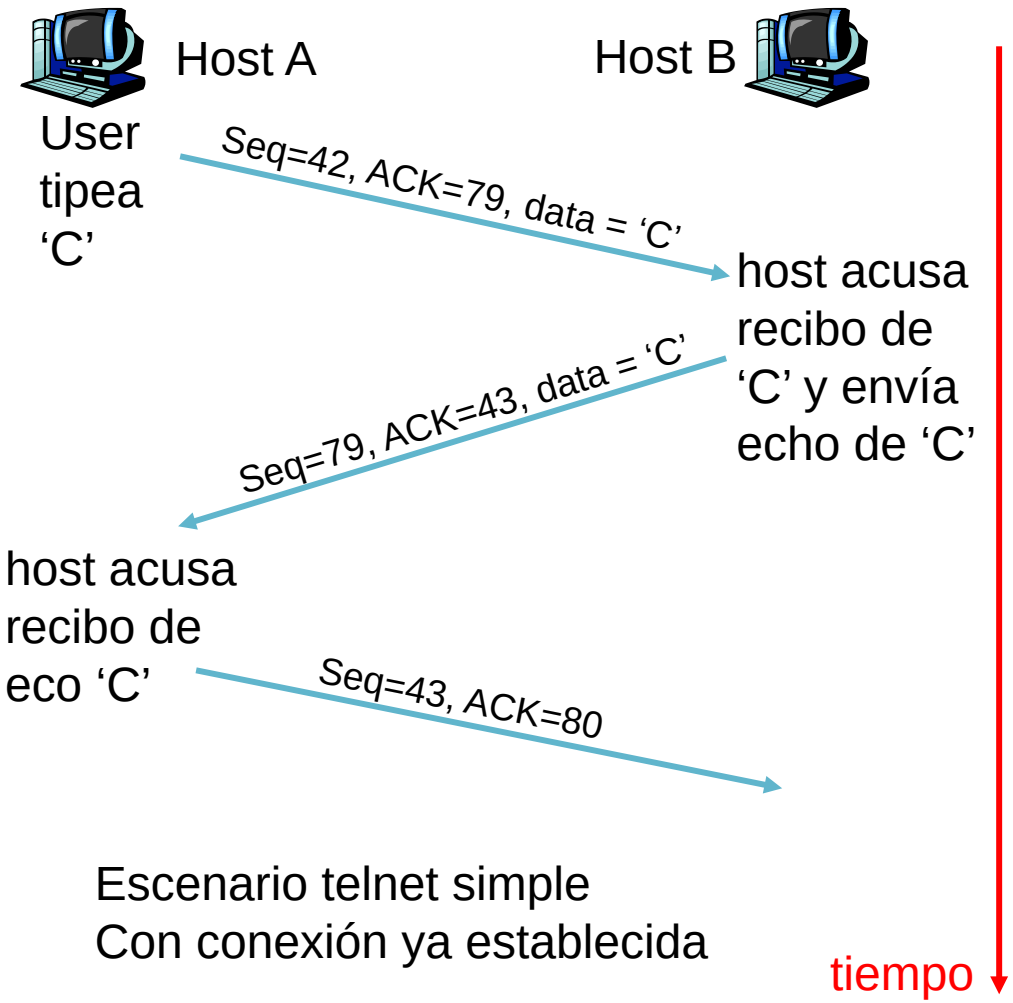
Estructura de un Segmento TCP



Número de Secuencia y ACKs en TCP

- **<u>Número de Secuencia:</u>**
  - "Número" del byte dentro del flujo correspondiente al primer byte del segmento de datos.
- **<u>ACKs:</u>**
  - Número de secuencia del próximo byte esperado desde el otro lado.
  - El ACK es acumulativo.
- ¿Cómo el receptor maneja segmentos fuera de orden?

  La especificación de TCP lo deja a criterio del implementador.



Escenario telnet simple
Con conexión ya establecida

Round Trip Time (RTT) y Timeout en TCP

¿Cómo fijar un valor de timeout en TCP?

- Tiene que ser mayor que el RTT, pero el RTT varía:
  - Si es **muy corto,** habrá **timeouts prematuros** y tenemos retransmisiones innecesarias.
  - Si es **muy largo, lenta reacción a pérdidas** de segmentos.

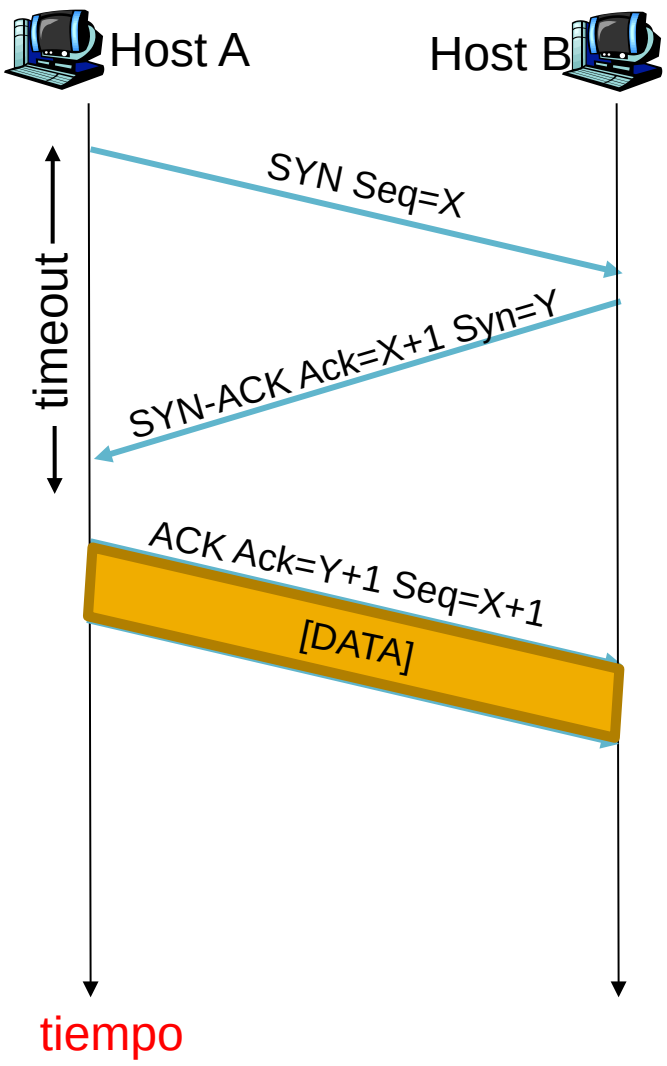
- ***¿Cómo estimar el RTT (Estimated RTT)?***
  - SampleRTT mide tiempo desde la transmisión del segmento hasta recibo del ACK. Ignora retransmisiones.
  - SampleRTT varía, hay que suavizar el RTT estimado. Hay que promediar varias medidas recientes, no considerar sólo el último SampleRTT.

Administración de Conexión en TCP

INICIO DE LA CONEXIÓN

Saludo de manos de tres vías (Three Way Handshake):

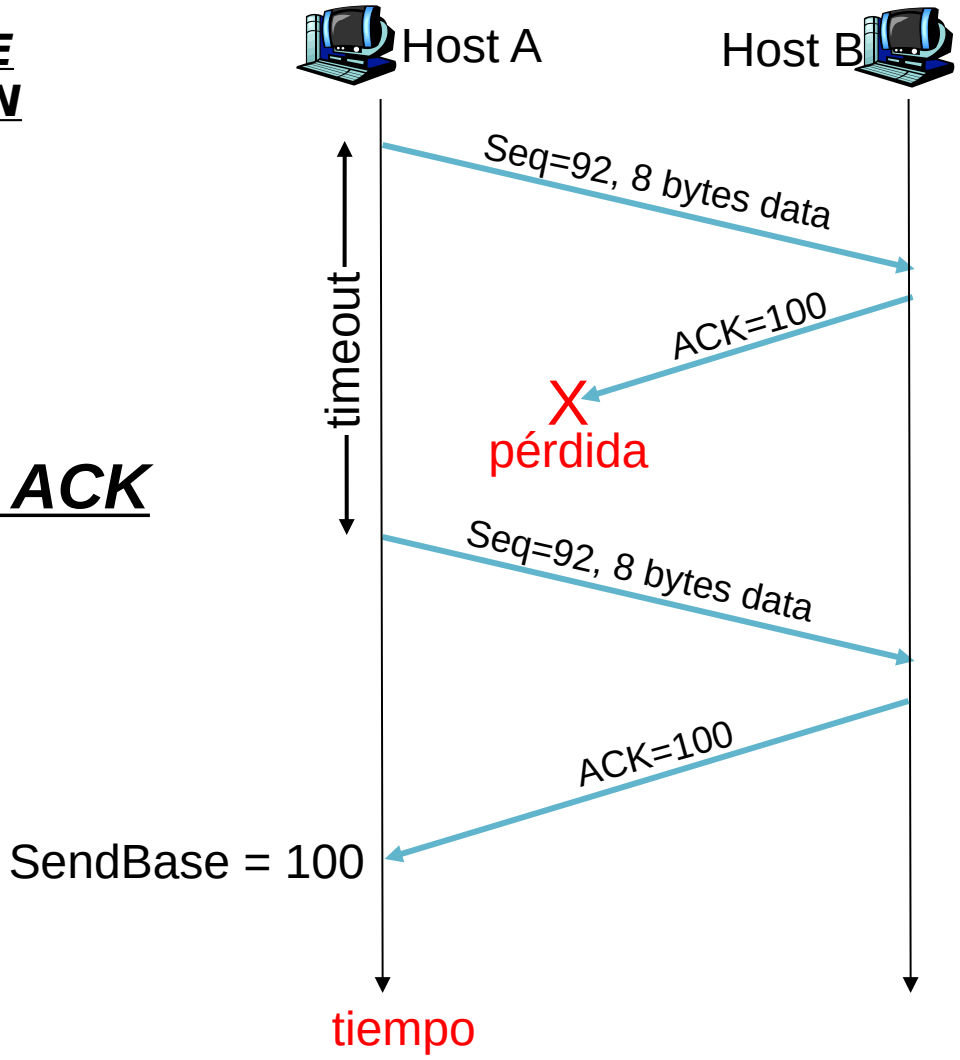
- **Paso 1:** el host cliente envía segmento TCP SYN al servidor.
  - Especifica el *Número de Secuencia* inicial.
  - Sin datos.
- **Paso 2:** host servidor recibe SYN, responde con segmento SYN & ACK.
  - Servidor ubica buffers.
  - Especifica el *Número de Secuencia* inicial.
- **Paso 3:** el cliente recibe SYN & ACK, responde con segmento ACK, el cual *ya podría* contener datos.



Administración de Conexión en TCP

ESCENARIOS DE RETRANSMISIÓN

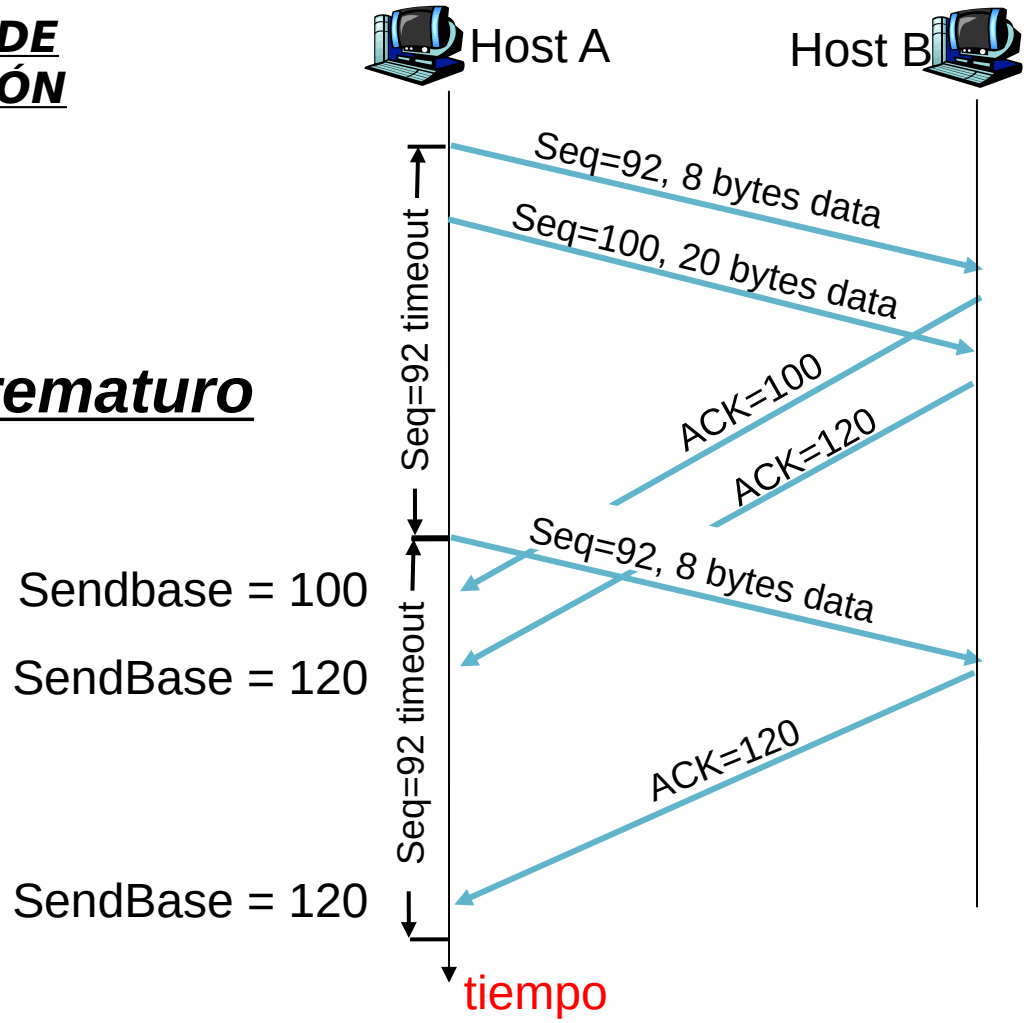
Pérdida de ACK



Administración de Conexión en TCP

ESCENARIOS DE RETRANSMISIÓN

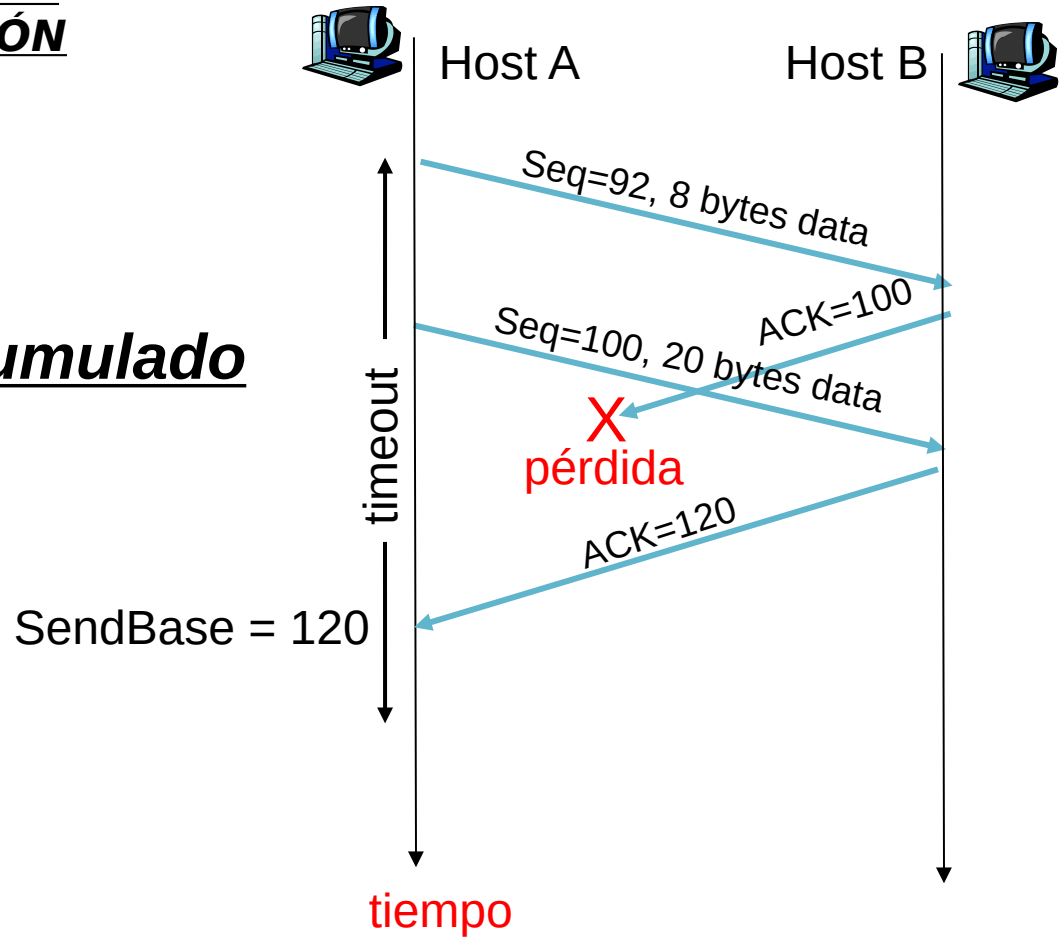
Timeout Prematuro



Administración de Conexión en TCP

ESCENARIOS DE RETRANSMISIÓN

ACK Acumulado



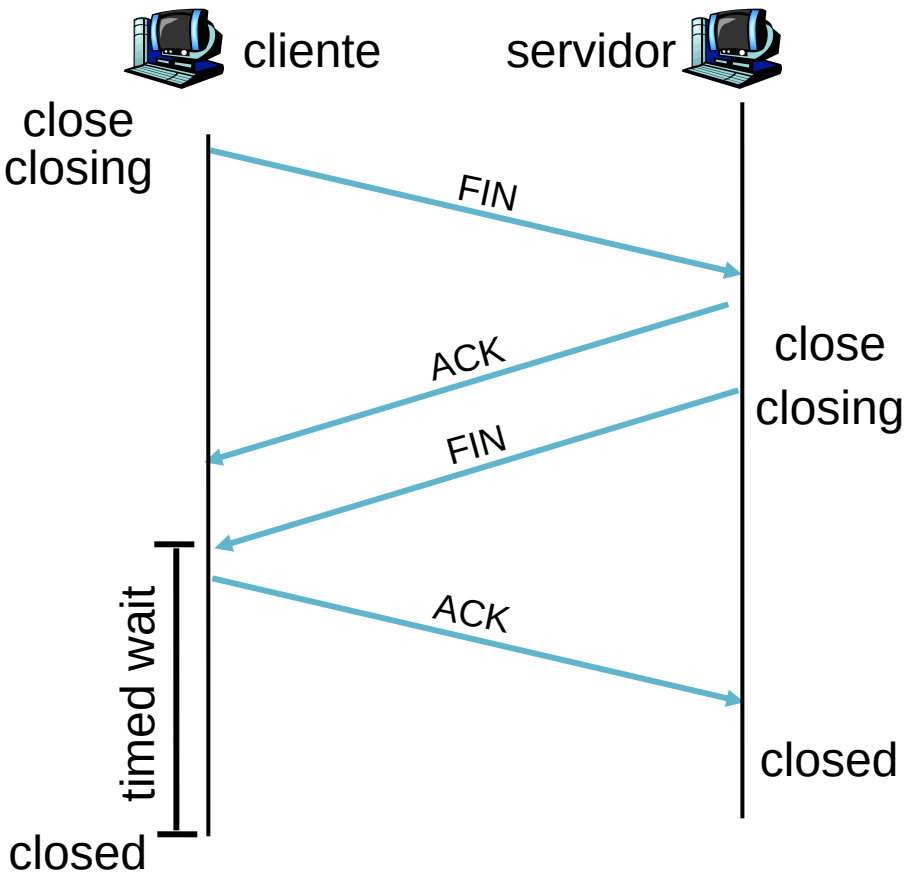
Administración de Conexión en TCP

CLIENTE CIERRA LA CONEXIÓN

Ciente cierra socket:

```
clientSocket.close();
```

- **Paso 1:** el host cliente envía el segmento TCP FIN al servidor.
- **Paso 2:** servidor recibe FIN, responde con ACK. Ante un cierre de conexión de la aplicación y envía FIN.
- **Paso 3:** el cliente recibe FIN, responde con ACK. Entra en "tiempo de espera" y responderá con ACK a FINs recibidos.
- **Paso 4:** servidor, recibe ACK. Pasa a conexión cerrada.



Control de Congestión en TCP

- Usa control extremo a extremo (sin asistencia de la red).
- El transmisor limita su volumen de transmisión:

  `LastByteSent - LastByteAcked ≤ min{CongWin, RcvWindow}`
- Aproximadamente:

$$\text{tasa} = \frac{\text{CongWin}}{\text{RTT}} \quad \text{Bytes/sec}$$

- **`CongWin`** es dinámica y función de la congestión percibida de l

¿Cómo percibe la congestión el transmisor?

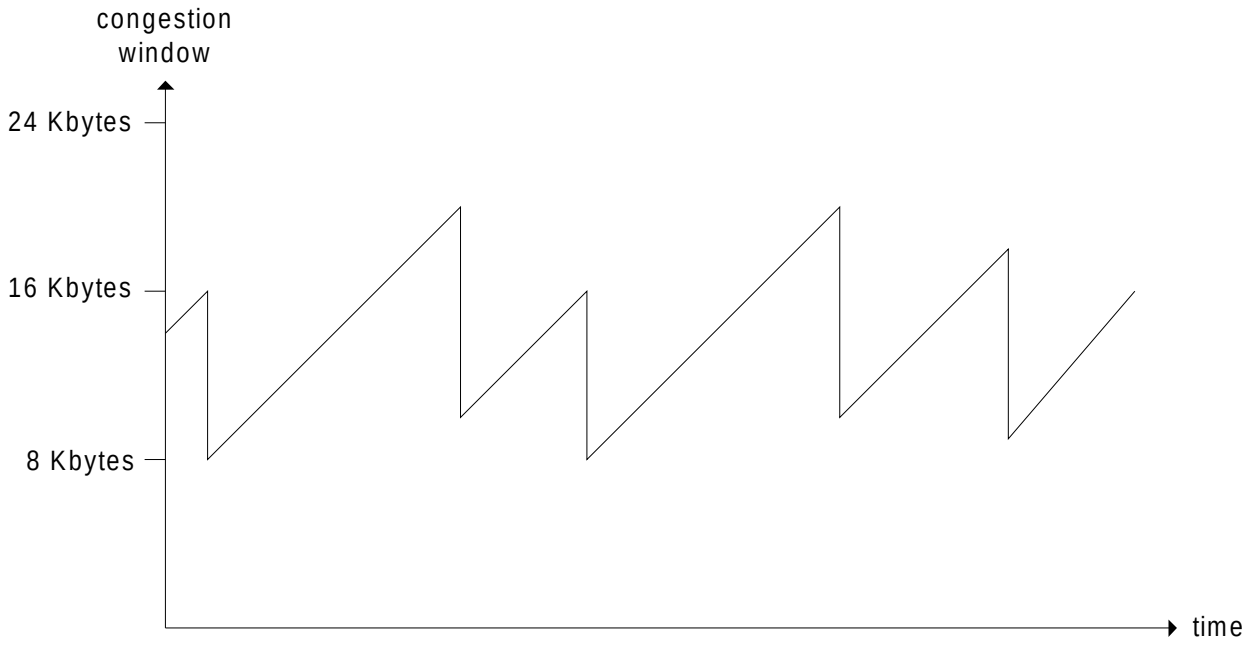
- Pérdidas (timeout ó 3 ACKs duplicados).

¿Qué hace cuando detecta las pérdidas?

- El transmisor TCP reduce su tasa (**`CongWin`**) después de pérdida
- Hay tres mecanismos:
  - AIMD (Additive-Increase, Multiplicative-Decrease).
  - Partida lenta.
  - Conservativo Después de Timeout.

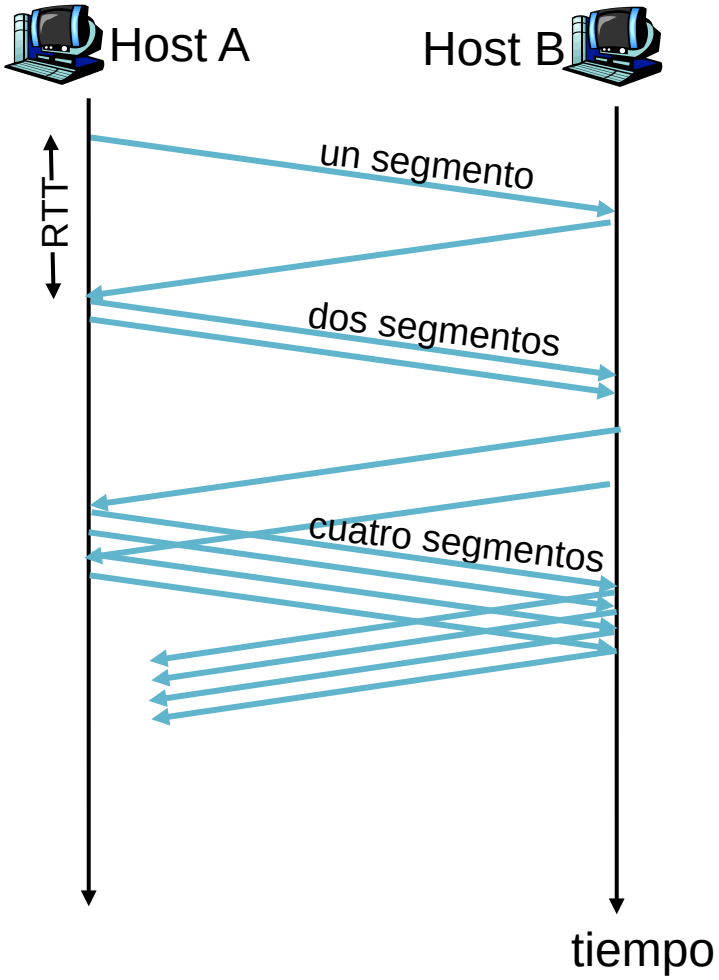
AIMD en TCP

- ***Aumento aditivo:*** se incrementa aumenta **`CongWin`** en 1 MSS cada RTT en ausencia de pérdida. En algunas implementaciones **`CongWin`** incrementa en MSS (MSS/CongWin) por cada ACK.
- ***Decrecimiento multiplicativo:*** reducir CongWin a la mitad luego de pérdida.



Partida Lenta en TCP

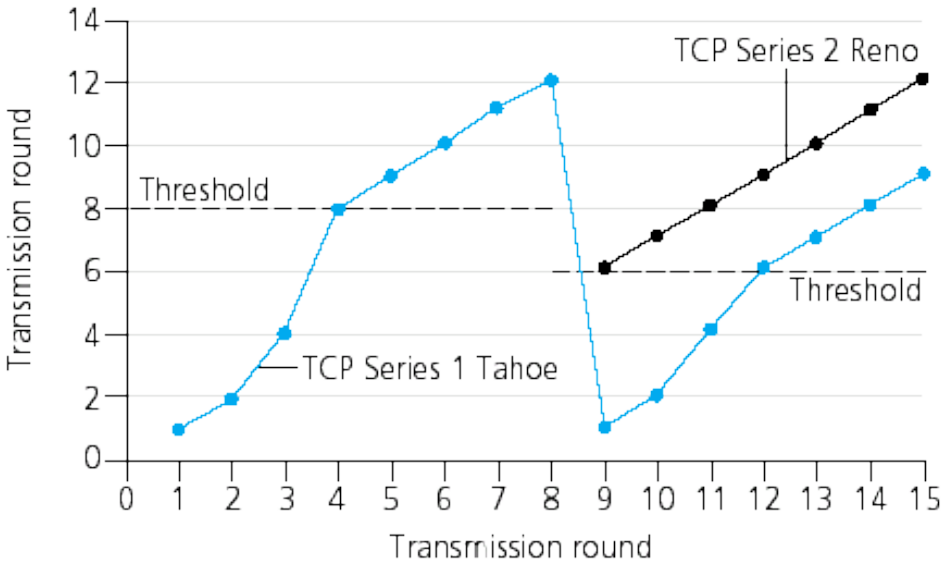
- Cuando la conexión comienza, aumenta la tasa exponencialmente hasta primera pérdida.
- La tasa inicial es lenta pero se acelera exponencialmente rápido.



Conservativo Después de Timeout

Filosofía:
- 3 ACKs duplicados indican la red es capaz de transportar algunos segmentos.
- Timeout antes de 3 duplicados es “más alarmante”.

- Después de 3 ACKs duplicados:
  - **`CongWin`** baja a la mitad.
  - Luego la ventana crece linealmente.
- Pero luego de un timeout:
  - **`CongWin`** es fijada en 1 MSS.
  - La ventana crece exponencialmente.
  - Hasta un umbral, luego crece linealmente.



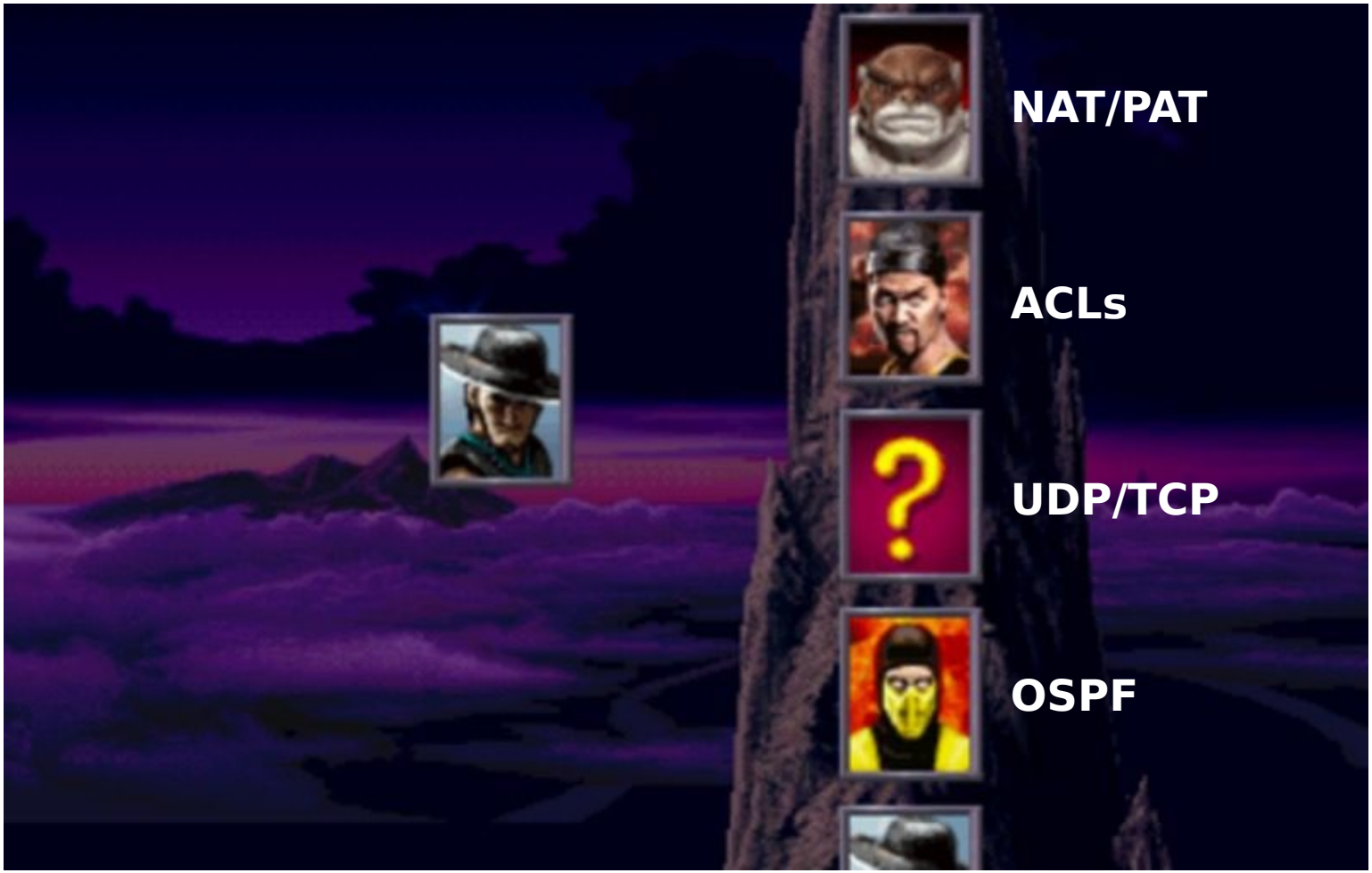
¿Por qué TCP es justo? - Equidad

Equidad y UDP

- Aplicaciones multimedia no usan TCP, no quieren tasa ahogada por control de congestión.
- En su lugar usan UDP. “Bombardean” audio/video a tasa constante y toleran pérdidas de paquetes.

Equidad y conexiones TCP paralelas

- Nada previene a las aplicaciones de abrir conexiones paralelas entre dos hosts. Los navegadores web hacen esto.
- Ejemplo: Sea un enlace de tasa R soportando 9 conexiones:
  - Una aplicación nueva pide 1 conexión TCP, obtendrá R/10.
  - Si la aplicación nueva pide 11 conexiones TCP, obtendrá R/2.

Suficiente de este tema…

Próxima Ronda…